Distilled Command Parser on a Local Kubernetes Cloud

Cloud Computing (CS3081), Mini Project Report

Manas Singh | Roll no. 230103017 | Section B | IIIT Senapati, Manipur

Repository: https://github.com/Singh-Manas-Web/cloud-kd-parser (final commit tagged `final`)

Overview

A 70M-parameter student (EleutherAI/pythia-70m) was trained by **sequence-level knowledge distillation** on replies from the teacher **phi3.5** (3.8B, Ollama). It turns a short English command into a four-key JSON object (intent, date, time, place). The student is served by a FastAPI container on a single-node k3d cluster. Datasets and model files live in SeaweedFS, an S3-compatible object store; the image contains no weights, and every Pod downloads the model at start-up.

Environment. Windows 11 laptop, 8 GB RAM, NVIDIA GTX 1650 (4 GB), Ubuntu on WSL2, Docker Desktop 4.98, k3d 5.9 with k3s v1.35.5, Python 3.11, CPU-only PyTorch 2.5.1 for training and inference. Every number below comes from a file in `results/`; anything that is not measured is marked as an assumption.

Data flow: commands_pool.jsonl in SeaweedFS (bucket datasets) → 2 labelling Jobs in namespace pipeline → phi3.5 on the laptop (host.k3d.internal:11434) → shard-0/shard-1.jsonl → clean.py → datasets/v1/train.jsonl → train.py (laptop CPU) → bucket models/v1, v2 and GitHub Releases → Deployment parser in namespace app (HPA 1–2) behind a ClusterIP Service and a Traefik Ingress on localhost:8080.

1. Object Storage and Parallel Teacher Labelling

Cluster and storage. `cluster/k3d.yaml` creates one server node with the Ingress on port 8080 and the SeaweedFS S3 NodePort on 8333. SeaweedFS runs in namespace *data* on a 5 GiB PVC; buckets *datasets* and *models* were created and commands_pool.jsonl was uploaded. The S3 keys are generated at deploy time into an untracked `.env` and stored only in Kubernetes Secrets.

Teacher. phi3.5 (Q4_0) served by Ollama 0.40.14 on the laptop GPU, temperature 0, seed 42, JSON output mode, at most 96 new tokens. The prompt (`prompts/teacher_prompt.txt`) lists the eight intents, the three slot rules, and eight few-shot examples taken from the handout, none from validation.jsonl. Replies are stored exactly as generated, in `teacher_raw`.

Sharding (≤ half page). Each worker reads the pool from object storage and keeps the lines whose line number *i* (counting from 0) satisfies *i* mod SHARD_COUNT = SHARD_INDEX. Two Kubernetes Jobs ran at the same time with SHARD_INDEX 0 and 1 (SHARD_COUNT 2), so the shards are disjoint and together cover every line, with no coordination between workers. Each worker writes only its own object, datasets/teacher/shard-<index>.jsonl, so there are no write conflicts. S3 objects cannot be appended to, so a worker re-uploads its object every 20 records and, if it restarts, downloads it first and skips ids already done (resumable). Both workers shared one GPU; each reported about 4.4 s per command, and both Jobs completed in 110 min with 0 restarts.

Measured. clean.py found all 3,000 pool ids in the shards (0 missing, 0 repeated, 0 unknown). Each record is counted under the first rule it fails; duplicates are then removed from the survivors, keeping the smallest id. SHA-256 of train.jsonl: `140f4508d0c882df8a7bacc0ed11bd2ef791c56859b43db963c655a9af82dbc5`. Re-running clean.py on the same shards reproduces this value.

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) bad slot	Duplicates	Kept
3,000	4	1	0	117	48	2,830

Check: 4 + 1 + 0 + 117 + 48 + 2,830 = 3,000. The 117 slot rejections are mostly values the teacher changed or invented, such as "dale'sinbirthday" for *when is dale's birthday*.

Limitations and improvements. The labels are unchecked teacher output (117 replies broke the slot rules), SeaweedFS keeps a single copy on one PVC, and the workers depend on the teacher running on the laptop. Improvements: hand-check a random sample of labels, use an Indexed Job so the shard index is assigned automatically, and run replicated object storage.

2. Training the Student

Recipe. Each example is `Command: <text>\nJSON: <teacher JSON><eos>`, with the loss computed on the JSON part only (the prompt tokens are masked). Training uses the first 1,500 records of the cleaned dataset (C0001–C1570) with the settings in `training/config.yaml`: Pythia-70M at revision step143000, AdamW, learning rate 4e-4 with 5% warm-up and linear decay, batch 16, maximum length 96, weight decay 0.01, gradient clip 1.0, seed 42, 10 epochs. v2 is the same recipe with 11 epochs. `make train SUBSET=50` trains on the first 50 records. Inference is greedy and stops at the first `}`, with at most 64 new tokens.

Artefacts. The model, tokenizer, version.json and model.sha256 (sha256sum format) are in bucket models (v1/, v2/) and in GitHub Releases model-v1 and model-v2. Downloading model-v1 and running `sha256sum -c` gave OK for all 7 files (results/task2/release_checksum.txt). No weights are committed to Git.

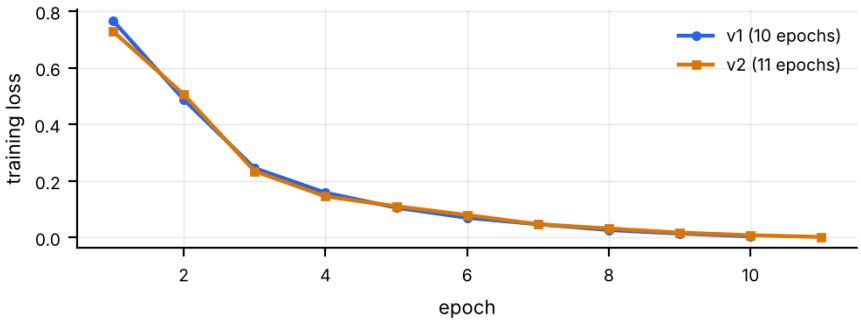


Figure 1. Training loss per epoch (training/loss.csv, loss_v2.csv). v1 fell from 0.767 to 0.0042 in about 38 min on the CPU; v2 fell from 0.729 to 0.0020.

Measured. Validation set (100 commands), scored with tools/score.py; raw outputs and reports are in results/.

Model	JSON valid	Intent	Date	Time	Place	Full frame
Teacher phi3.5 (same prompt and settings)	100	90	79	87	89	52
Untrained Pythia-70M	0	0	0	0	0	0
Student v1 (Pythia-70M, distilled)	100	88	81	82	86	47
Student v2 (one more epoch)	99	88	79	86	81	46

What changed. The untrained model never produced a JSON object; it continued the text. After distillation, v1 is valid on every validation command and reaches 47% full frame, which is 90% of the teacher's 52% (the requirement is at least half). The student is even slightly better on dates (81 vs 79), probably because training discarded the teacher replies that broke the slot rules. It is weaker on time (82 vs 87), and it repeats teacher habits such as "six am" for *at six*. One more epoch (v2) did not help: validity fell to 99% and full frame to 46%, a sign of over-fitting at a training loss of 0.002.

Sequence-level vs logit-based distillation (≤ 1 page). (a) *Sequence-level* distillation trains the student with ordinary cross-entropy on the *text the teacher generated*: the teacher's output sequence is treated as the target label. (b) *Logit-based* (word-level) distillation instead trains the student to match the teacher's *probability distribution over the vocabulary* at every token position, usually by minimising a temperature-softened KL divergence; these "soft targets" also carry information about the alternatives the teacher considered. (c) It is not possible here for two reasons. Ollama returns only the generated text, not the full next-token distribution. And phi3.5 and Pythia-70M use different tokenizers and vocabularies (about 32k Llama-style tokens vs about 50k GPT-NeoX tokens), so the two distributions are over different token sets and cannot be compared position by position. (d) *Limitation:* the student learns only the teacher's single greedy answer and has no gold labels, so it inherits the teacher's systematic errors, and the teacher's 52% full frame is a ceiling. With 1,500 short examples and a 70M model, it also over-fits quickly, as v2 shows.

Limitations and improvements. Only 1,500 of the 2,830 cleaned records were used (as specified), and no early stopping was applied. Improvements: keep a held-out split for early stopping (v1 at 10 epochs already beat v2 at 11), train on all cleaned records, and correct the teacher's systematic slot errors before training.

3. The Service in a Container

FastAPI service (`service/app.py`): `POST /parse` returns only the four-key object, with headers X-Model-Version and X-Served-By (the Pod name from the downward API); `GET /healthz` returns 200 while the process is alive; `GET /readyz` returns 503 until the model is loaded. Configuration comes only from the environment: MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS and FAULT_INJECT. Every output is checked against schema.json; an invalid output, or FAULT_INJECT=1, returns HTTP 500 `{"error": "invalid_model_output"}`. The image (python:3.11.11-slim, CPU-only torch 2.5.1, all packages pinned) runs as a non-root user with one torch thread, matching the 1-CPU limit.

Measured. Image size 1.58 GB on disk (335 MB compressed); limit 2 GB. With `--cpus=1 --memory=1g` (confirmed by docker inspect: NanoCpus=1e9, Memory=1,073,741,824), 50 sequential requests all returned HTTP 200 in 0.42–0.60 s each (median 0.47 s). Peak memory was **573.4 MiB** (docker stats sampled every second), and OOMKilled=false. /readyz returned 503 {"status":"loading"} for about 15 s, then 200. FAULT_INJECT=1 returned 500 with the exact error body. Evidence: results/task3/.

Limitations and improvements. Each Pod generates one reply at a time (a lock around generate), and most of the 1.58 GB image is PyTorch. /parse has no authentication. Improvements: export to ONNX or int8 to cut image size and

latency, add request batching, and put an API key or mTLS in front of the service.

4. Deployment, Scaling and Reliability on Kubernetes

Namespace *app*: Deployment *parser* with 2 replicas; requests 500m CPU / 512 Mi; limits 1 CPU / 1 Gi; liveness probe on /healthz, readiness probe on /readyz; rolling update with maxSurge 1 and maxUnavailable 0; preStop sleep of 10 s so the Ingress stops routing before a Pod exits; S3 keys from a Secret. A ClusterIP Service, a Traefik Ingress for /parse, and an HPA (min 1, max 2, 70% of the CPU request, scale-down window shortened to 60 s). The Locust load in every experiment used 4 users through the Ingress; the evidence for each is in results/task4/<experiment>/.

Experiment	Result (measured)
Readiness	After the Pod was deleted, the new Pod was 0/1 for about 19 s and the Service had no endpoints; at 1/1 the endpoint 10.42.0.16:8000 appeared (endpoints_during_load.txt).
Autoscaling	Under load, HPA CPU rose to 109% and then 196% of target; event "New size: 2; cpu above target". Both Pods then ran at up to 1,000m. After the load stopped (05:48:44) CPU fell to 0%, and by 05:50:21 the HPA was back to 1 replica ("New size: 1; All metrics below target"). The 4-minute load in this test: 602 requests, 0 failed.
Rolling update	Under a 6-minute load: MODEL_VERSION/MODEL_URI changed v1→v2 at 05:52:06 (X-Model-Version: v2 at 05:53:25), then kubectl rollout undo at 05:53:27 (X-Model-Version: v1 at 05:54:36). Requests 929, failed 1 (**0.11%**, limit 0.5%); both versions answered requests during the update (headers.json).
Recovery	Under a 3-minute load, Pod parser-…-cqqwc was deleted at 05:58:18; Kubernetes created parser-…-jhbzb and the Deployment returned to 2/2 Ready. Requests 492, failed 0 (**0%**, limit 2%).
Cold start	All Pods deleted 3 times; the logs show each new Pod downloading s3://models/v1/ (285.3 MB in 4.8–5.7 s). Creation→Ready: 20, 24 and 30 s; **median 24 s**.
Load balancing	Exactly two Ready replicas, standard command: 316 requests, 0 failed; parser-…-djx9w served 158 (49.8%) and parser-…-5fjs6 served 159 (50.2%).

Notes. The single rollout failure was an HTTP 500 from output validation (rollout_failures.csv: CatchResponseError('HTTP 500')), not a refused connection. That is consistent with v2's 1% invalid outputs, but the specific request was not logged, so this is an interpretation. The rollout and recovery summary files use the generic heading "2 min"; their real load durations were 6 and 3 minutes, and the request counts are the full runs.

Limitations of the local cluster. The cluster is a **single node** inside one laptop, so it does not demonstrate fault tolerance across machines: if the laptop, WSL or Docker fails, every replica and SeaweedFS fail together. Both replicas share the same 8 CPU threads, so scaling to 2 doubles the CPU allowance but cannot add hardware. metrics-server samples about every 15 s, so the HPA reacts with a delay of tens of seconds. SeaweedFS has one replica on a local-path PVC, with no redundancy. The 60 s scale-down window was chosen to keep the test short; the default is 300 s.

Improvements. A multi-node cluster with Pod anti-affinity and a PodDisruptionBudget; caching the model on a shared volume (or in an init container) to shorten the 24 s cold start; and HPA scaling on request rate or latency rather than CPU alone.

5. Networking: Subnets and a Network Access Control List

Usable hosts = $2^{32-\text{prefix}} - 2$. Allocated largest first from 10.20.0.0/16, each subnet starting right after the previous one (working in network/subnets.md).

Subnet	Hosts needed	Network	Prefix	Usable range	Broadcast	Usable hosts
app	1,000	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

NACL for the public subnet (network/nacl.json). Inbound: 90 deny all from 198.51.100.0/24; 100 allow TCP 80 and 110 allow TCP 443 from 0.0.0.0/0; 120 allow TCP 22 from 10.20.5.128/27; 130 allow TCP 1024–65535 from 10.20.0.0/22 (replies from the app servers). Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024–65535 to 0.0.0.0/0 (replies to clients). Anything else is denied by default. **Measured:** nacl_sim.py --check gives 6/6 correct on the sample packets (results/task5/nacl_check.txt).

Why reply rules and rule order matter (≤ half page). A NACL is *stateless*: it does not remember that it allowed a connection, so every packet, including a reply, is checked against the rules on its own. A client's request to port 443 is allowed inbound, but the response leaves the subnet towards the client's ephemeral port (1024–65535) and needs its own outbound rule. Likewise, the app servers' replies (source port 8000) arrive at the load balancer's ephemeral ports and need an inbound rule. A stateful security group would allow these replies automatically. Rules are evaluated from the lowest number upward, and *the first match decides*. A packet from 198.51.100.9 to port 443 matches both the deny for 198.51.100.0/24 and the allow for 443 from 0.0.0.0/0, so the deny must carry a smaller number (90) than the allow

rules (100, 110). Numbered above them, the allow would match first and the abusive range could still reach the web ports.

Limitations and improvements. Stateless rules must open the whole ephemeral range (1024–65535) for replies, which is broad. Improvements: pair the NACL with stateful security groups on the instances, and narrow the ephemeral range if the operating systems allow it.

6. Testing and CI/CD

`.github/workflows/ci.yml` runs four stages in order: (1) lint (ruff) plus a gitleaks scan of the whole Git history; (2) 9 unit tests (pytest: clean.py rejection rules, de-duplication and counts, schema validation, /healthz, /readyz, response headers and FAULT_INJECT returning 500); (3) model evaluation, which downloads the model from a GitHub Release, verifies model.sha256, generates predictions for validation.jsonl and scores them; (4) a quality gate, `score.py --min-validity 95`. A manual trigger takes the Release tag to evaluate. `make up` builds and deploys everything from a fresh clone (fetching the models from the Releases); `make down` deletes the cluster.

Measured. Run on model-v1 passed all four stages: actions/runs/42341969740. The deliberately bad model (untrained Pythia-70M, Release model-bad) passed lint, tests and evaluation, then **failed at the quality gate**: actions/runs/42341958991. gitleaks reported no leaks. **Fresh-clone check:** after `make down` ("Successfully deleted cluster kd!"), `git clone`, `cd` and `make up` rebuilt everything in about 10 minutes: the cluster, SeaweedFS and buckets, both models fetched from the Releases with every checksum OK, the image built and deployed (2/2 Ready), ending with HTTP 200 and X-Model-Version: v1 (results/task6/make_up_fresh_clone.txt).

Problems met. The first gitleaks-action run crashed on the first push, because it scans from the parent of the root commit, which does not exist. It was replaced by the gitleaks CLI scanning the whole history. Before the Releases existed, evaluation failed at the download step, as expected.

Limitations and improvements. CI checks the model but does not build the image or deploy it, and the actions are pinned by tag rather than by commit hash. Improvements: build and scan the image in CI (for example with Trivy), run a smoke deployment on a throwaway kind or k3d cluster, and pin the actions by SHA.

7. Performance, Cost and Review

Measured. Standard command (4 users, spawn rate 4, 2 min, through the Ingress, two Ready replicas). Files: results/loadtest_stats.csv, loadtest_headers.json, task4/lb/top_pods.txt.

Metric	Value
p50 / p95 latency	1,500 ms / 2,100 ms (limit p95 ≤ 5,000 ms)
Throughput	2.65 requests/s (316 requests in 120 s)
Error rate	0.00% (0 of 316)
CPU per Pod (kubectl top, every 5 s)	parser-…-5fjs6: avg 955m, max 999m; parser-…-djx9w: avg 965m, max 1,001m (both at the 1-CPU limit)
Memory per Pod	parser-…-5fjs6: avg 556 Mi, max 611 Mi; parser-…-djx9w: avg 547 Mi, max 611 Mi
Cold start (Task 4)	median 24 s (20, 24, 30 s)

Cost of 1,000 requests (rate_card.json: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; each replica billed for its limits, 1 vCPU and 1 GiB):

```
one replica per hour = 1 × 3.50 + 1 × 0.45 = 3.95 INR
all running replicas = 2 × 3.95 = 7.90 INR per hour
requests served per hour = 2.6536 req/s × 3,600 = 9,552.9
cost per 1,000 requests = 7.90 / 9,552.9 × 1,000 = 0.827 INR
```

SLO. Target: 99% of /parse requests return HTTP 200 within 30 s. Window: one standard load-test run. Error budget = 1% × 316 requests = 3.16, so 3 failed requests are allowed. Used: 0 failed requests = 0% of the budget. Across the other runs: rolling update 1 of 929 (budget 9.29, 11% used), recovery 0 of 492, HPA 0 of 602.

Review (every evidence cell is measured):

Component	Evidence	Limitation found	Improvement proposed
Data and storage	2,830 of 3,000 kept; 0 missing, 0 repeated ids; 117 slot rejections	Labels are unchecked teacher output; SeaweedFS has one replica on one disk	Hand-check a labelled sample; run replicated object storage across nodes
Model quality	v1: 100% valid, 47% full frame (teacher 52%)	Inherits the teacher's slot errors; time accuracy 82% vs teacher's 87%	Use all 2,830 records, early stopping on a held-out split, a stronger teacher prompt
Latency and throughput	p95 2,100 ms; 2.65 req/s with 2 replicas	One request at a time per Pod (generation lock); CPU-bound	Int8 quantisation or ONNX Runtime; dynamic batching; more replicas

Component	Evidence	Limitation found	Improvement proposed
Resource utilisation	about 960m CPU (at the limit) and at most 611 Mi of 1 Gi per Pod	CPU saturated while about 40% of memory is unused	Lower the memory limit to 768 Mi; raise the CPU per Pod or the HPA maximum
Reliability and recovery	Rollout 0.11% failed; recovery 0% failed; cold start median 24 s	Single node: no protection against machine failure; 24 s to replace a Pod	Multi-node cluster, PodDisruptionBudget, a model cache on a shared volume
Security and cost	0.827 INR per 1,000 requests; gitleaks: 0 leaks	Static S3 keys; plain HTTP; no authentication on /parse; replicas billed when idle	Rotate keys and use short-lived credentials; TLS and an API key at the Ingress; scale to zero when idle

Assumptions. The cost assumes the two replicas run for the whole hour at the measured throughput and are billed for their limits; no real cloud charge was incurred. All timings were measured on one laptop that was also running Docker Desktop, WSL2 and the browser, so absolute latencies would differ on dedicated hardware.

Use of AI tools

Claude (Anthropic) was used to explain the assignment and to draft the code, manifests and this report's structure. I ran every step on my own laptop, checked the outputs, fixed the problems that came up (WSL, Docker permissions, the k3d API address, network resets, the gitleaks first-push error), and took all numbers from my own result files. Details are in AI_USE.md.